

Find the Missing Character

Contents

The Problem.....	1
The Computational Solution	1

The Problem

In our collection, each lynching event is typically described in multiple newspaper sources. The different sources often provide different details, different spellings of names, etc. It would be useful to rely on a single summary of an event that takes into account contradictory details; this would allow us to grasp a story in a single reading, and in all its different nuances. We used the collection of newspaper articles on each event to compile by handⁱ often contradictory narratives into a single narrative “compilation”.ⁱⁱ Different spellings of names of individuals and locations, different dates and times, different actors and actions were weaved into a single narrative with attribution to the specific source. Thus, for Event ID 261 (the lynching of Grant Welley), all five available newspaper articles (e.g., *The Atlanta Constitution* of 9-4-1900) contain the person’s name as Fleming but the compilation has the name Flemming (2 m’s); the *Thomasville Weekly Times-Enterprise* of 9-8-1900 reports the person’s name as Welly, contrary to all newspaper articles and to the compilation; the same newspaper article of the *Thomasville Weekly Times-Enterprise* reports the location Coolidge, contrary to the location Cooledge found in the other four articles and in the compilation.

The same problem arises for compilers of folk stories, as many different versions of the same traditional folk tale (e.g., Jacobs’ compilation of English folk tales, 1890 or the Grimm brothers’ collection of German folktales, 2014 [1812, 1857]). Which characters, which characters’ attributes or actions are spurious? The Grimm brothers agonized over these questions for years about each of the stories. But, returning to our corpus of newspaper articles, we used a basic rule in manually compiling event narratives: all different social actors (individuals, groups, organizations), and all their different actions mentioned in any of the available newspaper sources must be inserted into the compilation. Upon close inspection, though, compilations typically missed some actors and the role they played because of their infrequent occurrence (e.g., church ministers, souvenir hunters, a judge or a white friend of the negro, using the newspaper language of the time, who would address the mob). The problem persisted even after all compilations were inspected and edited by different researchers, multiple times. It became clear that with a few hundred events and several thousand newspaper articles an automatic strategy for checking the reliability of compiled stories was required.

The Computational Solution

This Python 3 script takes the list of social actors obtained from WordNet (*Find the character* tool) and checks the values in the list against a list of words found in both summaries and original sources. To obtain the latter list, we use the parser you selected in the NLP Suite setup: Stanford CoreNLP (Manning et al. 2014; <https://stanfordnlp.github.io/CoreNLP/>), Stanza (Qi et al. 2020), or spaCy (Honnibal et al. 2020). The parser tags each word (token) in both source document and compilation for its Part of Speech value (POSTAG, for nouns, verbs, adjectives, adverbs, etc.). We focus on POSTAG values for nouns: NN, noun singular, NNP, proper noun singular, NNS, noun plural, and NNPS, proper noun plural. We then check each of these nouns to see if they are in the WordNet social actor list. Any social actor in a newspaper source not detected in the compilation’s social actor list, is flagged as “missing.”

The parser also tags each word for its NER value (Named Entity Recognition). Of particular interest here are the NER values of Location, Date, Person, Organization (other values, such as Numbers, Percentages, etc. are also recognized, but they are not of interest to us in this context). We then check each of these NER values in the different sources for each event against the NER values of the compilation. Again, any NER value in a newspaper source not detected in the compilation, is flagged as “missing.” The output in csv format lists all cases of error, organized in five rows (one for each type of error: social actor, NER person, organization, date, and location) and six columns: Type of Error, Frequency of Summaries in Error, Percentage of Summaries in Error, Frequency of Error, List of Summary Filenames for Type of Error, List of Documents for Type of Error.ⁱⁱⁱ

Figure # – validity_freq.csv file output of frequencies of compilation errors by error type (missing social actors and missing NER values)

Figs. # -# visualize the frequencies of missing social actors and missing NER values in Excel column and pie charts; the two exported lists of compilation and source filenames for each type of error (social actors and NER values) are displayed when hovering over each column (see Fig. # and #).

Figure # – Column chart of frequencies of compilation errors by error type (missing social actors and missing NER values)

Figure # – Pie chart of frequencies of compilation errors by error type (missing social actors and missing NER values)

ⁱ Current computational technology makes it possible to create such summaries automatically — generated by a computer algorithm rather than by hand. The NLP Suite offers this through its BERT summarizer tool. On automatic text summarization more generally, see Gambhir and Gupta 2017; Lloret and Palomar 2012; Nenkova and McKeown 2012.

ⁱⁱ We use the word “compilation”, rather than “summary”, since, by and large, we maintained the original newspaper language (e.g., the word “negro”, rather than “African American”) and original story line, however contrived the story may have appeared to be.

ⁱⁱⁱ The column “List of Documents for Type of Error” may be split in several columns depending upon the number of documents found in error.